École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes
- Rabat

Projet Data Engineering

# Plateforme de Data Warehouse Streaming pour l'Analyse des Ventes

**Élèves :**
Anass Bensmina
Zakariae Bellil
Aymane El Akkioui
Moubarak Benaqqa

**Encadrant :**
Meryem El Fakhouri

**Jury :**

Année Universitaire 2025-2026

# Introduction générale

Dans un contexte économique où le volume des transactions numériques croît de façon exponentielle, la capacité à collecter, transformer et analyser des données en temps réel est devenue un avantage concurrentiel déterminant. Les entreprises e-commerce génèrent chaque seconde des flux de données relatifs aux commandes, aux paiements et aux comportements clients, données dont la valeur analytique est immense mais souvent sous-exploitée faute d'une infrastructure adaptée.

Ce projet a pour ambition de concevoir et de réaliser une plateforme complète de Data Warehouse orientée streaming, capable d'ingérer des données transactionnelles en temps quasi-réel, de les transformer selon une architecture en médaillon (Bronze, Silver, Gold), puis de les exposer à des outils de visualisation et à des modèles de Machine Learning pour en extraire des insights à haute valeur ajoutée.

La source de données choisie est le jeu de données public *Olist* — une marketplace brésilienne de e-commerce — qui fournit un ensemble réaliste de commandes, articles, paiements, avis et informations géographiques portant sur plus de 100 000 transactions.

Ce rapport s'articule autour de quatre chapitres principaux. Le premier chapitre présente le cadrage du projet, la problématique et les choix technologiques. Le deuxième chapitre détaille la spécification des besoins fonctionnels et non fonctionnels. Le troisième chapitre expose la conception architecturale et les modèles de données. Enfin, le quatrième chapitre est consacré à la réalisation technique, au pipeline de données et aux modules d'analyse avancée.

# Table des matières

**Introduction générale** **1**

**1 Cadrage du projet** **4**
1.1 Contexte et problématique . . . . . 4
1.2 Source de données . . . . . 4
1.3 Choix du modèle de développement . . . . . 5
1.3.1 Architecture orientée streaming . . . . . 5
1.3.2 Architecture en médaillon (Medallion Architecture) . . . . . 5
1.3.3 Justification des choix technologiques . . . . . 6

**2 Spécification des besoins** **7**
2.1 Spécification générale . . . . . 7
2.1.1 Besoins fonctionnels . . . . . 7
2.1.2 Besoins non fonctionnels . . . . . 7
2.2 Spécification détaillée . . . . . 8
2.2.1 Cas d'utilisation principaux . . . . . 8
2.2.2 Flux de données de bout en bout . . . . . 8

**3 Conception** **10**
3.1 Introduction . . . . . 10
3.2 Architecture Globale du Système . . . . . 10
3.2.1 Le Paradigme ELT : Puissance de Calcul In-Situ . . . . . 10
3.2.2 Découplage Total via Redpanda . . . . . 11
3.2.3 Séparation des Flux OLTP et OLAP . . . . . 11
3.2.4 Déroulement du Pipeline de Données . . . . . 11
3.3 Fondements Théoriques et Choix de ClickHouse . . . . . 12
3.3.1 La Dichotomie OLTP vs OLAP . . . . . 12
3.3.2 Le Stockage Orienté Colonnes . . . . . 12
3.3.3 Le Problème des Jointures et la Solution des Dictionnaires . . . . . 12
3.4 L'Architecture Medallion (Pipeline ELT) . . . . . 13
3.4.1 La Couche Bronze : Ingestion Brute . . . . . 13
3.4.2 La Couche Bronze : Ingestion Brute et Single Source of Truth . . . 13
3.4.3 La Couche Silver : Nettoyage et Enrichissement . . . . . 15
3.5 Modélisation Dimensionnelle : Le Schéma en Étoile . . . . . 16
3.5.1 La Table de Faits (`fact_order_items`) . . . . . 16
3.5.2 Les Tables de Dimensions . . . . . 16
3.6 Conception de la partie Machine Learning . . . . . 17
3.6.1 Conception du module de prévision des ventes . . . . . 17

3.6.2 Conception du module de segmentation des clients . . . . . 17
3.6.3 Conception du module de détection des anomalies . . . . . 18
3.6.4 Choix d'une architecture modulaire . . . . . 18
3.7 Conclusion . . . . . 18

**4 Réalisation** **19**
4.1 Environnement et outils de développement . . . . . 19
4.1.1 Stack technique et topologie . . . . . 19
4.1.2 Déploiement et Infrastructure as Code . . . . . 20
4.2 Implémentation du Producer : Simulation Transactionnelle . . . . . 20
4.3 Implémentation des Consumers : Routage et Persistance . . . . . 20
4.3.1 Consumer ClickHouse (Voie OLAP) . . . . . 20
4.3.2 Consumer PostgreSQL (Voie OLTP) . . . . . 21
4.4 Implémentation du pipeline ELT (Medallion Architecture) . . . . . 21
4.4.1 Transformation Bronze vers Silver (Nettoyage) . . . . . 21
4.4.2 Transformation Silver vers Gold (Modélisation Dimensionnelle) . . . 21
4.5 Réalisation de la partie Machine Learning . . . . . 22
4.5.1 Chargement et préparation des données . . . . . 22
4.5.2 Réalisation du *feature engineering* . . . . . 23
4.5.3 Réalisation du module de prévision des ventes . . . . . 24
4.5.4 Réalisation du module de segmentation des clients . . . . . 24
4.5.5 Réalisation du module de détection des anomalies . . . . . 24
4.5.6 Sauvegarde des modèles et intégration dans le data warehouse . . . 25
4.5.7 Bilan de la réalisation . . . . . 25
4.6 Visualisation et Restitution (Metabase) . . . . . 26
4.6.1 Vue Globale (Executive) . . . . . 26
4.6.2 Performance des Ventes . . . . . 27
4.6.3 Performance Opérationnelle . . . . . 30
4.6.4 Limitations identifiées et plan de remédiation . . . . . 31

**Conclusion Générale** **33**

# Chapitre 1

# Cadrage du projet

## 1.1 Contexte et problématique

Les entreprises du commerce électronique font face à un défi fondamental : leurs systèmes transactionnels (OLTP) sont optimisés pour l'écriture rapide, mais inadaptés à l'analyse. Exécuter des requêtes analytiques complexes directement sur une base PostgreSQL de production dégrade les performances opérationnelles et retarde la prise de décision.

Par ailleurs, les événements de vente se produisent en continu. Une architecture batch traditionnelle, qui traiterait les données une fois par nuit, introduit une latence inacceptable pour des cas d'usage comme la détection de fraude en temps réel, le suivi des niveaux de stock ou l'analyse des tendances d'achat horaires.

La problématique centrale de ce projet est donc la suivante : **comment concevoir une infrastructure de données capable d'ingérer des flux transactionnels en temps quasi-réel, de les transformer de manière incrémentale, et de les exposer à des outils d'analyse et de Machine Learning, tout en maintenant la disponibilité du système opérationnel ?**

## 1.2 Source de données

Le projet s'appuie sur le jeu de données **Olist**, un dataset public issu d'une marketplace brésilienne de e-commerce, disponible sur Kaggle. Il couvre la période 2016–2018 et contient :

- **100 000+ commandes** avec leurs statuts, horodatages et identifiants clients
- **Lignes d'articles** avec produit, vendeur, prix et frais de livraison
- **Paiements** avec type, montant et nombre d'échéances
- **Avis clients** avec score (1–5) et commentaires textuels
- **Données géographiques** avec codes postaux, villes, États et coordonnées GPS
- **Catalogue produits** avec catégories en portugais et traduction anglaise

Ce dataset présente l'avantage d'être suffisamment riche pour illustrer des cas d'usage analytiques réels (segmentation géographique, prévision de ventes, détection d'anomalies) tout en étant maîtrisable dans un environnement de développement local.

## 1.3 Choix du modèle de développement

### 1.3.1 Architecture orientée streaming

Le choix d'une architecture orientée streaming, plutôt que d'un ETL batch classique, répond à plusieurs impératifs :

— **Latence réduite :** Les données sont disponibles dans le Data Warehouse quelques secondes après leur génération, contre plusieurs heures avec un batch nocturne.

— **Scalabilité horizontale :** Un broker Kafka peut absorber des millions d'événements par seconde en ajoutant des partitions et des consumers.

— **Découplage des systèmes :** Le producer (source opérationnelle) et les consumers (destinations analytiques) évoluent indépendamment sans interdépendance directe.

— **Rejoue des événements :** En cas d'erreur dans un consumer, il est possible de rejouer les messages depuis n'importe quel offset.

### 1.3.2 Architecture en médaillon (Medallion Architecture)

L'organisation des données en trois couches distinctes — Bronze, Silver, Gold — est un pattern reconnu dans l'industrie (popularisé par Databricks/Delta Lake) qui apporte :

— **Bronze :** Données brutes ingérées telles quelles, sans transformation. Sert de source de vérité immuable et permet la rejouabilité.

— **Silver :** Données nettoyées, enrichies (ex : géolocalisation), dédupliquées et typées correctement. Prêtes pour l'analyse exploratoire.

— **Gold :** Données modélisées en schéma en étoile, agrégées et optimisées pour la performance des requêtes analytiques et des dashboards.

### 1.3.3 Justification des choix technologiques

| **Composant** | Technologie choisie | Justification |
|---|---|---|
| **Message Broker** | Redpanda | Compatible Kafka, plus léger, pas de JVM, idéal pour le dev local |
| **Base OLTP** | PostgreSQL 15 | Standard de l'industrie, excellente gestion des FK et contraintes |
| **Base OLAP** | ClickHouse | Moteur colonnaire haute performance, dictionnaires pour lookups O(1) |
| **Orchestration** | Apache Airflow | Standard de facto pour les pipelines data, DAGs en Python |
| **Visualisation** | Metabase | Interface no-code pour les utilisateurs métier, dashboards interactifs |
| **ML** | scikit-learn | Bibliothèque mature, modèles classiques bien adaptés au use case |
| **Conteneurisation** | Docker Compose | Reproductibilité totale de l'environnement en une commande |

Table 1.1 – Choix technologiques du projet

# Chapitre 2

# Spécification des besoins

## 2.1 Spécification générale

### 2.1.1 Besoins fonctionnels

- **Ingestion en streaming :** Le système doit simuler un flux continu d'événements transactionnels (commandes, articles, paiements, avis) à partir de données historiques, en respectant l'ordre chronologique des transactions et en maintenant un état de reprise en cas d'interruption.
- **Consommation multi-destinations :** Les messages produits doivent être consommés simultanément par deux consumers indépendants : l'un persistant les données dans PostgreSQL (OLTP) avec gestion des contraintes de clé étrangère, l'autre dans ClickHouse (OLAP) avec insertion par lots pour la performance.
- **Transformation par couches :** Le pipeline doit transformer les données brutes de la couche Bronze vers la couche Silver (enrichissement géographique, nettoyage, traduction des catégories) puis vers la couche Gold (construction de la table de faits et des dimensions en schéma en étoile).
- **Orchestration automatisée :** Les transformations Bronze→Silver→Gold doivent être orchestrées et schedulées automatiquement via Apache Airflow, avec gestion des dépendances entre tâches et mécanisme de retry.
- **Analyse prédictive :** La plateforme doit embarquer un module de Machine Learning comprenant : la prévision temporelle des ventes (forecasting), la segmentation des clients (clustering RFM), et la détection des transactions anormales (anomaly detection). Les résultats doivent être réintégrés dans ClickHouse.
- **Visualisation interactive :** Un outil de BI doit permettre aux utilisateurs métier d'explorer les données Gold via des dashboards interactifs sans écrire de SQL.
- **Chargement initial des dimensions statiques :** Le système doit pré-charger les tables de référence (clients, produits, vendeurs, géolocalisation) dans les deux bases au démarrage, avant que le streaming commence.

### 2.1.2 Besoins non fonctionnels

- **Performance :** Le consumer ClickHouse doit insérer les données par lots de 500 lignes et flusher toutes les 10 secondes pour équilibrer latence et débit. Les requêtes analytiques sur la table de faits (100 000+ lignes) doivent s'exécuter en moins de 500 ms grâce à l'engine colonnaire de ClickHouse.
- **Fiabilité et at-least-once delivery :** Les consumers ne commitent les offsets Kafka qu'après une insertion réussie, garantissant qu'aucun message ne soit perdu

en cas d'erreur. Le consumer PostgreSQL embarque un mécanisme de dead-letter queue pour les enregistrements qui échouent après 10 tentatives.

- **Résilience :** Tous les services implémentent des boucles de retry avec backoff exponentiel pour la connexion aux brokers et bases de données. Les healthchecks Docker Compose garantissent le démarrage ordonné des services.
- **Reproductibilité :** L'intégralité de l'environnement doit être déployable en une seule commande `docker compose up`. Aucune configuration manuelle post-démarrage ne doit être nécessaire.
- **Observabilité :** Les consumers doivent journaliser chaque batch inséré avec le nombre de lignes et la table cible. Le producer doit journaliser chaque chunk streamé avec son horodatage.
- **Maintenabilité :** Les transformations SQL (Bronze→Silver, Silver→Gold) doivent être externalisées dans des fichiers `.sql` indépendants, versionnables et modifiables sans redéploiement de l'image Airflow.

## 2.2 Spécification détaillée

### 2.2.1 Cas d'utilisation principaux

Le système distingue deux types d'acteurs : le **Data Engineer** qui configure et exploite le pipeline, et l'**Analyste Métier** qui consomme les résultats.

| UC | Acteur | Cas d'utilisation | Description |
|---|---|---|---|
| UC1 | Data Engineer | Démarrer le pipeline | Lancer tous les services via Docker Compose |
| UC2 | Data Engineer | Monitorer le streaming | Vérifier les offsets et le débit via Redpanda Console |
| UC3 | Data Engineer | Déclencher les transformations | Activer le DAG Airflow pour Bronze→Silver→Gold |
| UC4 | Data Engineer | Exécuter les modèles ML | Lancer les scripts de training et d'inférence |
| UC5 | Analyste Métier | Consulter les dashboards | Explorer les KPIs de vente sur Metabase |
| UC6 | Analyste Métier | Analyser les segments clients | Visualiser les clusters RFM dans Metabase |
| UC7 | Analyste Métier | Suivre les prévisions | Consulter les forecasts de ventes |

Table 2.1 – Cas d'utilisation du système

### 2.2.2 Flux de données de bout en bout

Le flux de traitement suit un chemin déterministe à cinq étapes :

1. **Simulation :** Le seeder charge les données statiques (clients, produits, vendeurs, géolocalisation, traductions) dans PostgreSQL et ClickHouse au démarrage.
2. **Production :** Le producer lit les fichiers CSV Olist, trie les commandes chronologiquement, et publie des chunks de 100 commandes (avec leurs articles, paiements et avis associés) dans quatre topics Redpanda toutes les 60 secondes.
3. **Consommation :** Deux consumers lisent simultanément les quatre topics. Le consumer PostgreSQL insère dans l'OLTP en respectant l'ordre FK (ordres avant articles). Le consumer ClickHouse accumule les messages dans des buffers par topic et flush par batch.
4. **Transformation :** Le DAG Airflow exécute toutes les minutes les scripts SQL de transformation Bronze→Silver puis Silver→Gold.
5. **Analyse :** Les modèles ML consomment les données Gold depuis ClickHouse, produisent des prédictions, et les réinsèrent dans des tables dédiées (`ml_sales_forecast`, `ml_customer_segments`, `ml_order_anomalies`).

# Chapitre 3

# Conception

## 3.1 Introduction

La conception d'une plateforme de données moderne nécessite de concilier des exigences de haute disponibilité, de faible latence et de traitement analytique massif. Ce chapitre détaille l'architecture de notre système, de l'ingestion transactionnelle à l'exposition des données pour l'Intelligence Artificielle et la Business Intelligence. Notre réflexion s'appuie fortement sur Le cours de l'informatique décisionnelle, ainsi que les paradigmes détaillés par Martin Kleppmann dans son ouvrage *Designing Data-Intensive Applications*, notamment concernant la séparation des charges de travail OLTP (Online Transaction Processing) et OLAP (Online Analytical Processing).

## 3.2 Architecture Globale du Système

L'architecture globale de notre projet a été pensée pour être distribuée, scalable, découplée, et surtout, **orientée temps réel**. Au lieu de surcharger une base de données unique avec des requêtes transactionnelles et analytiques, ou de se limiter à des traitements par lots (batch) traditionnels et différés, nous avons opté pour une approche événementielle continue (*Event-Driven & Streaming Architecture*).

L'architecture de notre plateforme repose sur ce paradigme moderne de traitement de données à grande échelle, permettant de capter, transporter et analyser l'information à la milliseconde près, dès sa création. Avant de détailler les étapes de ce flux continu, il est crucial de justifier les choix structurants qui régissent la circulation des données en temps réel au sein du système.

### 3.2.1 Le Paradigme ELT : Puissance de Calcul In-Situ

Contrairement aux architectures traditionnelles basées sur l'ETL (*Extract, Transform, Load*), nous avons privilégié une approche **ELT** (*Extract, Load, Transform*). Dans un flux ETL, la transformation est effectuée par un moteur intermédiaire avant l'ingestion, ce qui crée souvent un goulot d'étranglement. En choisissant l'ELT, nous exploitons la puissance de calcul massivement parallélisée de ClickHouse. Les données sont ingérées le plus rapidement possible dans leur format brut (couche Bronze), laissant au moteur de la base de données le soin de réaliser les transformations lourdes (nettoyage, agrégations) lors des phases Silver et Gold. Cette approche garantit une meilleure scalabilité et une flexibilité totale : si les règles métier changent, les données brutes sont déjà présentes et prêtes à être retraitées sans ré-ingestion.

### 3.2.2 Découplage Total via Redpanda

Un principe fondamental de notre design est le **découplage strict entre producteurs et consommateurs**. L'intégration de Redpanda (compatible Kafka) comme bus d'événements central permet d'isoler le rythme de production des données de celui de leur consommation.

— **Résilience** : Si le Data Warehouse ClickHouse subit une maintenance, le producteur continue d'émettre des messages que Redpanda stocke de manière persistante.

— **Agilité** : Plusieurs consommateurs peuvent lire le même flux de données à des vitesses différentes et pour des objectifs variés (un pour l'ingestion analytique, un autre pour l'inférence ML en temps réel) sans jamais interférer les uns avec les autres.

### 3.2.3 Séparation des Flux OLTP et OLAP

Il est important de préciser que notre base **PostgreSQL** joue ici le rôle de simulateur de production. Dans un environnement réel, PostgreSQL gérerait les transactions de l'application web (gestion des sessions, inventaire). Bien que nous y stockions les états applicatifs pour simuler un écosystème complet, cette base est **structurellement isolée** du pipeline décisionnel. Aucun calcul analytique n'est effectué sur PostgreSQL afin de ne pas impacter les performances transactionnelles, respectant ainsi le principe de séparation des responsabilités entre les systèmes opérationnels (OLTP) et décisionnels (OLAP).

### 3.2.4 Déroulement du Pipeline de Données

Le cycle de vie d'une donnée, de sa création à sa visualisation, se décompose en quatre étapes clés :

1. **Émission et Persistance Opérationnelle** : Le producteur Python génère des transactions e-commerce au format JSON. Ces événements sont simultanément envoyés vers Redpanda pour le flux analytique et vers PostgreSQL pour simuler la persistance d'état de l'application source.
2. **Transport et Ingestion Temps Réel** : Redpanda achemine les messages vers des consommateurs microservices. Ces derniers injectent les données brutes directement dans la table **Bronze** de ClickHouse. À ce stade, la donnée est une copie exacte de l'événement source, garantissant l'auditabilité du système.
3. **Orchestration et Raffinement (ELT)** : C'est ici qu'intervient **Apache Airflow**. Agissant comme le chef d'orchestre du système, il déclenche périodiquement des scripts SQL natifs au sein de ClickHouse. Ce processus transforme la donnée brute (Bronze) en donnée nettoyée (Silver), puis en structures multidimensionnelles optimisées (Gold). Cette étape inclut l'enrichissement via les dictionnaires en RAM pour éliminer les jointures coûteuses.
4. **Exploitation et Intelligence** : Les données finales présentes dans le schéma en étoile sont consommées par deux services distincts :
   — **Metabase** pour la Business Intelligence, permettant aux décideurs de visualiser les KPIs en temps réel.

— **ML Services** pour l'intelligence artificielle, utilisant les données raffinées pour l'entraînement de modèles ou l'inférence prédictive.

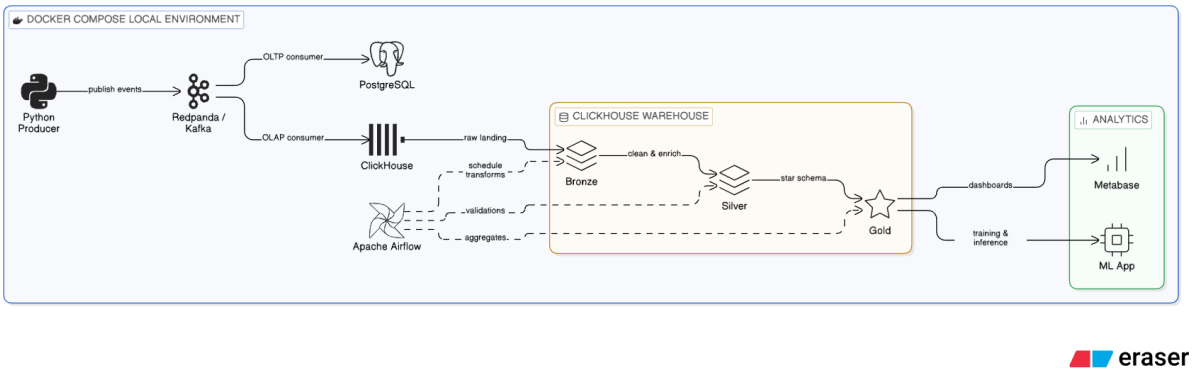


FIGURE 3.1 – Diagramme de l'Architecture Globale du Système

## 3.3 Fondements Théoriques et Choix de ClickHouse

### 3.3.1 La Dichotomie OLTP vs OLAP

Les modèles relationnels traditionnels basés sur les arbres B (B-Trees) sont optimisés pour les requêtes OLTP : des lectures et écritures rapides sur un petit nombre de lignes (par exemple, la validation d'une commande). Cependant, lorsqu'il s'agit d'analyser des millions de commandes pour calculer un chiffre d'affaires moyen, ces bases de données (PostgreSQL orienté lignes) montrent leurs limites à cause des lectures excessives sur disque.

### 3.3.2 Le Stockage Orienté Colonnes

Pour la partie analytique, notre choix s'est porté sur **ClickHouse**, un Data Warehouse open-source orienté colonnes. Dans un modèle orienté colonnes, toutes les valeurs d'une même colonne sont stockées de manière contiguë sur le disque. Cela permet une compression drastique des données et évite de charger en mémoire les colonnes inutiles lors des requêtes d'agrégation massives (`SUM`, `AVG`, etc.).

### 3.3.3 Le Problème des Jointures et la Solution des Dictionnaires

Les jointures dans les architectures distribuées orientées colonnes sont particulièrement coûteuses. Réaliser un *Hash Join* entre une table de faits de plusieurs milliards de lignes et plusieurs tables de dimensions oblige le moteur analytique à déplacer des quantités massives de données en mémoire, créant un goulot d'étranglement (bottleneck).

Pour contourner cette limitation inhérente aux bases orientées colonnes, notre conception exploite une fonctionnalité majeure de ClickHouse : **les Dictionnaires en RAM**. Au lieu de joindre physiquement les tables de dimensions (Clients, Produits, Vendeurs) à la table de faits à chaque requête, les dimensions sont chargées en mémoire vive de manière permanente sous forme de structure clé-valeur (Layout `COMPLEX_KEY_HASHED`).

Lors des requêtes décisionnelles, nous utilisons la fonction `dictGet()`, transformant une jointure SQL coûteuse en une simple recherche en mémoire en complexité algorithmique $O(1)$. Cela garantit des temps de réponse sous-millisecondes pour les tableaux de bord.

## 3.4 L'Architecture Medallion (Pipeline ELT)

Pour structurer notre Data Warehouse au sein de ClickHouse, nous avons mis en œuvre l'architecture Medallion, orchestrée par des DAGs Apache Airflow. Cette approche permet de garantir l'idempotence, la qualité des données et la traçabilité.

### 3.4.1 La Couche Bronze : Ingestion Brute

La couche Bronze représente l'historique brut des données. Les consommateurs Python connectés à Redpanda insèrent directement les événements JSON/CSV sous forme de flux (Append-Only) dans des tables `MergeTree`. Aucune transformation n'est appliquée à ce stade afin de préserver l'intégrité de la donnée source.

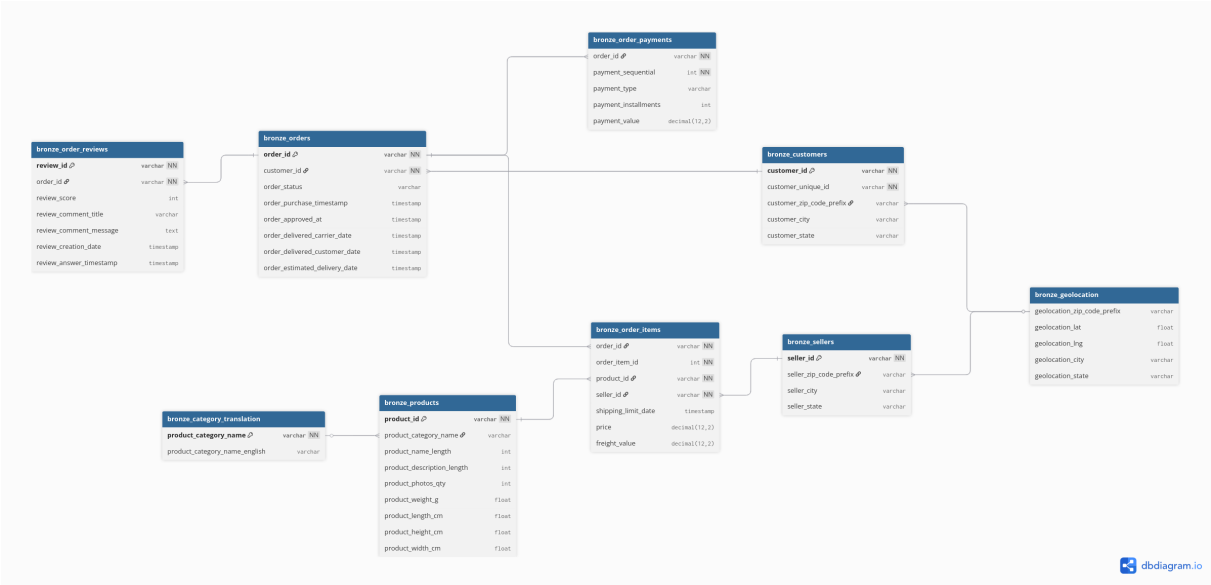


FIGURE 3.2 – Modèle de données de la couche Bronze

### 3.4.2 La Couche Bronze : Ingestion Brute et Single Source of Truth

La couche Bronze constitue le point d'entrée physique du Data Warehouse. Elle représente l'historique brut, inaltéré et exhaustif de toutes les données générées par le système source. Dans le paradigme de l'architecture Medallion, cette couche agit en tant que *Single Source of Truth* : elle garantit la traçabilité complète et offre la capacité de rejouer l'intégralité des pipelines de transformation (reprocessing) en cas d'évolution des règles métier dans les couches supérieures.

### Principes d'Ingestion et Immutabilité

L'ingestion des données dans la couche Bronze suit une logique stricte d'ajout seul (*Append-Only*). Aucune opération de modification (`UPDATE`) ou de suppression (`DELETE`) n'est exécutée à ce stade, afin de préserver l'intégrité absolue de la donnée telle qu'elle a été émise. Les données sont ingérées via deux flux distincts :

— **Flux Batch (Données de Référence)** : Les tables dimensionnelles statiques (clients, vendeurs, produits, géolocalisation) sont initialisées directement depuis les fichiers sources par le script *Unified DB Seeder*.

— **Flux Streaming (Données Transactionnelles)** : Les microservices consommateurs Python, souscrivant aux topics Redpanda, insèrent les événements transactionnels (commandes, paiements, avis) en temps réel dès leur création dans l'application OLTP.

### Conception Physique : Le Moteur MergeTree et Indexation

La modélisation de cette couche s'appuie exclusivement sur la famille de moteurs `MergeTree` de ClickHouse. Ce choix est fondamental pour supporter les taux d'insertion massifs du streaming tout en préparant les lectures analytiques. La structure des tables Bronze se caractérise par :

— **Typage permissif et miroir de la source** : Les types de données reflètent exactement ceux des systèmes transactionnels (majoritairement des `String`, `Float64`, `Int32` et `DateTime`). Les valeurs manquantes (`NULL`) ou les erreurs de formatage originelles (comme des fautes de frappe dans les catégories) sont conservées intentionnellement. Toute la logique de data quality est délibérément repoussée à la couche Silver.

— **Indexation primaire clairsemée (*Sparse Index*)** : Contrairement aux bases de données OLTP basées sur des B-Trees, ClickHouse organise physiquement les données sur le disque en blocs (granules) selon une clé de tri stricte définie par la clause `ORDER BY`. Par exemple, la table transactionnelle `bronze_orders` est triée chronologiquement (`ORDER BY order_purchase_timestamp`), tandis que `bronze_order_items` est triée par une clé composite (`ORDER BY (order_id, order_item_id)`). Cela garantit que les futures requêtes d'agrégation d'Airflow ne liront que les blocs de données strictement nécessaires.

### Cartographie des Tables Bronze

La modélisation physique de cette couche comprend un ensemble de tables calquant la topologie exacte du dataset e-commerce d'origine :

— **Entités transactionnelles (Streaming)** : `bronze_orders`, `bronze_order_items`, `bronze_order_payments`, `bronze_order_reviews`.

— **Entités descriptives (Batch/Seeding)** : `bronze_customers`, `bronze_sellers`, `bronze_products`, `bronze_category_translation`, `bronze_geolocation`.

En conclusion, la conception de la couche Bronze privilégie la résilience, la vitesse d'écriture en continu et la fidélité absolue à la source. C'est ce choix architectural d'immutabilité et de découplage qui permet au pipeline ELT (Extract, Load, Transform) d'être

robuste, déléguant toute la complexité des calculs (Jointures, Nettoyage, Agrégations de la table de faits) aux couches Silver et Gold orchestrées par Apache Airflow.

### 3.4.3 La Couche Silver : Nettoyage et Enrichissement

La couche Silver filtre, nettoie et consolide la couche Bronze. Orchestrée par Airflow via le `ClickHouseOperator`, cette couche effectue :

- La gestion des valeurs nulles (coalescence).
- Le mapping géographique (jointure des codes postaux avec les coordonnées de latitude/longitude).
- La traduction des catégories de produits (du portugais vers l'anglais).
- L'abandon des colonnes lourdes et non analytiques (comme les textes libres des commentaires clients).

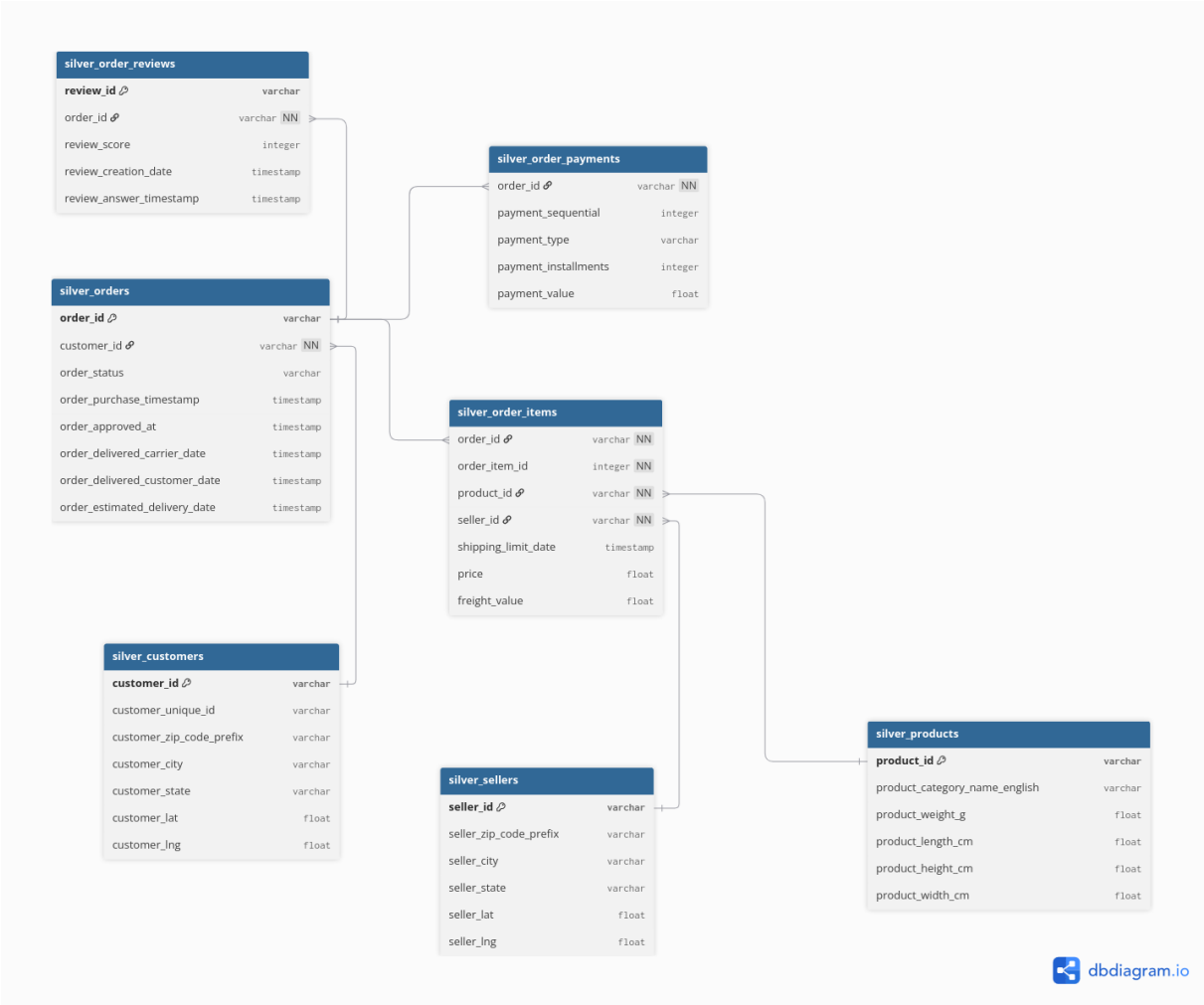


FIGURE 3.3 – Modèle de données de la couche Silver (Données filtrées)

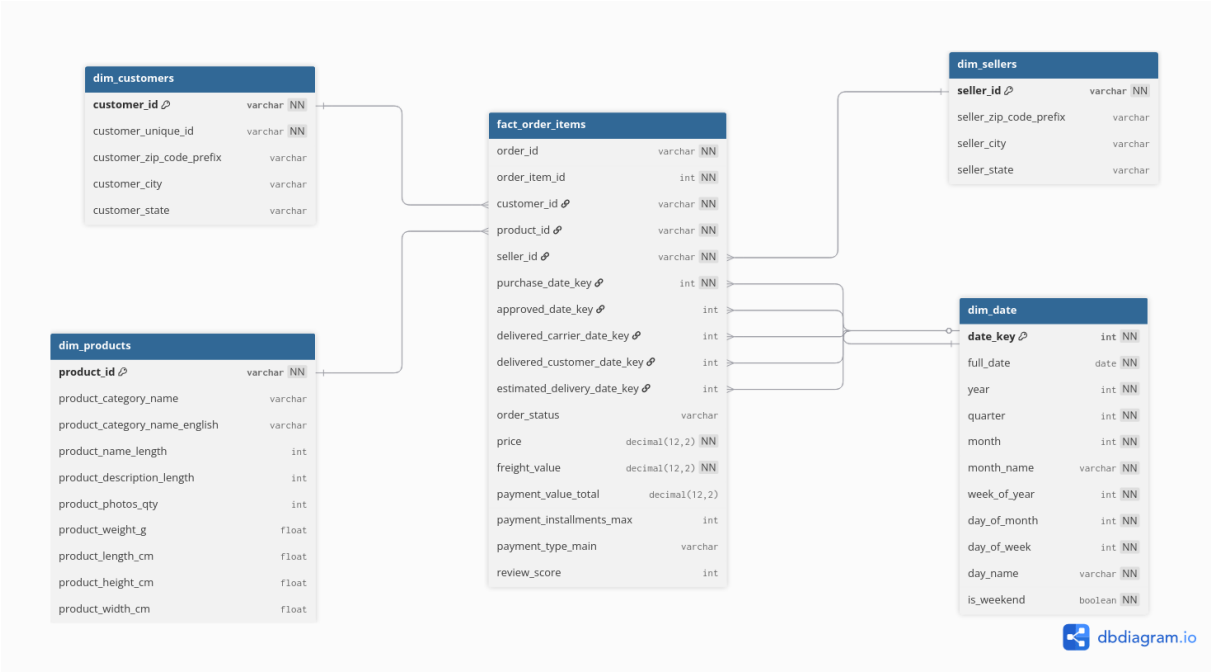


FIGURE 3.4 – Modèle de données de la couche Gold (Schéma en Étoile)

## 3.5 Modélisation Dimensionnelle : Le Schéma en Étoile

Pour la couche Gold, nous avons implémenté le modèle de conception classique : le schéma en étoile, adapté aux moteurs analytiques modernes.

### 3.5.1 La Table de Faits (`fact_order_items`)

Cette table centralise les métriques transactionnelles. Elle est extrêmement étroite (narrow table) et ne contient que des clés étrangères (Identifiants) et des mesures quantitatives (Prix, Frais de port, Scores d'évaluation). Le typage y est strict (ex : `Int32` au lieu de `DateTime` pour les dates clés) afin de maximiser la compression et la rapidité du moteur `MergeTree` de ClickHouse.

### 3.5.2 Les Tables de Dimensions

Les dimensions contiennent les attributs descriptifs de l'activité commerciale.

— **Dimensions Métier** : Clients, Vendeurs, et Produits. Bien que persistées physiquement dans la couche Silver, elles sont exposées à la couche Gold *via* les Dictionnaires en RAM évoqués précédemment.

— **Dimension Temps (`dim_date`)** : Une table générée de manière déterministe lors de l'initialisation du système (couvrant 10 années). Elle permet aux utilisateurs métier de réaliser des analyses temporelles (Saisonnalité, Ventes par Trimestre, Comportement Week-end vs Jour de semaine) sans nécessiter de calculs temporels coûteux à l'exécution.

## 3.6 Conception de la partie Machine Learning

La partie *Machine Learning* a été conçue comme une composante analytique avancée venant compléter l'architecture de streaming et de data warehouse du projet. Alors que les couches précédentes assurent la collecte, le stockage, la transformation et la visualisation des données, cette couche apporte une dimension supplémentaire orientée vers l'anticipation, la compréhension des comportements et l'aide à la décision. L'objectif principal de cette conception est donc de transformer les données historiques de vente en informations prédictives et exploitables par les décideurs.

Afin de répondre à cet objectif, la partie *Machine Learning* a été structurée autour de trois axes fonctionnels : la prévision des ventes, la segmentation des clients et la détection des anomalies. Ce découpage a été choisi pour couvrir des besoins complémentaires. La prévision permet d'anticiper les tendances futures, la segmentation permet de mieux comprendre les profils des clients, tandis que la détection des anomalies aide à repérer les comportements inhabituels ou exceptionnels dans les ventes.

### 3.6.1 Conception du module de prévision des ventes

Le module de prévision des ventes a été conçu dans le but d'estimer l'évolution future du chiffre d'affaires à partir des données historiques disponibles. Dans cette optique, le choix a été fait de travailler sur une granularité mensuelle. Cette décision se justifie par le fait que les données journalières peuvent contenir de nombreuses fluctuations ponctuelles, alors qu'une agrégation mensuelle fournit une vision plus stable et plus adaptée à un modèle de prévision.

La variable cible retenue est le revenu mensuel. Afin d'expliquer cette variable, plusieurs caractéristiques ont été définies. Parmi elles, on retrouve l'indice temporel, le numéro du mois, ainsi que des variables de retard représentant les revenus observés sur les périodes précédentes. Ce choix permet de capturer à la fois la dynamique chronologique de la série et l'influence des mois antérieurs sur les mois futurs.

Sur le plan méthodologique, le problème a été formulé comme une tâche de régression supervisée. Cette approche permet de comparer plusieurs algorithmes capables de modéliser la relation entre les caractéristiques temporelles construites et la variable de sortie. La conception du module prévoyait donc dès le départ une logique de comparaison entre plusieurs modèles afin de sélectionner celui qui donne les meilleures performances.

### 3.6.2 Conception du module de segmentation des clients

Le module de segmentation des clients a été pensé pour mieux analyser le comportement d'achat et distinguer différents profils au sein de la clientèle. L'idée centrale de cette partie est qu'un même ensemble de clients n'a pas tous les mêmes habitudes : certains achètent souvent, d'autres très rarement, certains dépensent davantage, alors que d'autres ont une contribution plus faible au chiffre d'affaires.

Pour représenter ces comportements, une approche basée sur des indicateurs de type *RFM* enrichi a été retenue. Les variables choisies sont la récence, qui mesure le temps écoulé depuis le dernier achat, la fréquence, qui reflète le nombre de commandes effectuées, le montant total dépensé, ainsi que la valeur moyenne des commandes. Ensemble, ces

variables permettent d'obtenir une représentation synthétique mais pertinente de l'activité de chaque client.

Comme il n'existe pas de catégories prédéfinies de clients dans les données, cette tâche a été conçue comme un problème d'apprentissage non supervisé. Le but n'est donc pas de prédire une classe connue à l'avance, mais de laisser l'algorithme identifier automatiquement des groupes cohérents de clients partageant des caractéristiques proches.

### 3.6.3 Conception du module de détection des anomalies

Le module de détection des anomalies a été conçu afin de repérer les variations inhabituelles du revenu. Dans un contexte décisionnel, ce type d'analyse est important car il permet d'identifier des journées qui se distinguent fortement du comportement habituel, qu'il s'agisse d'une baisse anormale, d'un pic exceptionnel ou d'un événement particulier.

Le choix a été fait de travailler à une échelle journalière, car cette granularité permet de détecter plus rapidement les changements inattendus. Deux approches ont été envisagées lors de la conception. La première est une approche statistique fondée sur le *Z-score*, choisie pour sa simplicité et sa facilité d'interprétation. La seconde est une approche d'apprentissage non supervisé avec *Isolation Forest*, retenue pour sa capacité à identifier des observations rares ou atypiques de manière plus flexible.

Cette double conception permet à la fois de disposer d'une méthode de référence simple et d'une méthode plus avancée, afin de comparer leur comportement sur les données réelles du projet.

### 3.6.4 Choix d'une architecture modulaire

Au-delà des trois modules, la partie *Machine Learning* a été globalement conçue selon une architecture modulaire. Chaque tâche possède son propre pipeline de préparation de données, son propre ensemble de variables, son propre modèle et ses propres résultats. Cette séparation présente plusieurs avantages : elle améliore la lisibilité du projet, facilite la maintenance du code et permet de faire évoluer chaque module indépendamment des autres.

Cette conception modulaire s'intègre également bien à l'architecture générale du projet. Les résultats produits par les modèles peuvent être sauvegardés, réutilisés puis intégrés dans le data warehouse, ce qui assure une continuité entre la couche de traitement des données et la couche d'analyse prédictive.

## 3.7 Conclusion

La conception de cette architecture démontre une compréhension approfondie de l'ingénierie des données à grande échelle. En dissociant les opérations (PostgreSQL/Redpanda) de l'analytique (ClickHouse), et en adoptant une modélisation ELT optimisée par des structures en mémoire, nous avons bâti un socle de données robuste. Ce socle garantit la réactivité nécessaire aux dashboards Metabase tout en fournissant des caractéristiques fiables pour l'entraînement continu de l'infrastructure MLOps.

# Chapitre 4

# Réalisation

Ce dernier chapitre présente la concrétisation technique du projet. Après avoir défini l'architecture conceptuelle et les modèles de données, nous détaillons ici l'implémentation effective de chaque composant, les choix d'ingénierie effectués lors du développement, et les résultats obtenus.

## 4.1 Environnement et outils de développement

### 4.1.1 Stack technique et topologie

La réalisation du projet s'appuie sur un écosystème entièrement conteneurisé, structuré autour des couches standards de l'ingénierie des données modernes :

- **Couche d'Application et Ingestion (Python 3.11) :** Développement du *Producer* et des *Consumers* natifs. Utilisation de la bibliothèque `confluent-kafka` (basée sur C/librdkafka), choisie pour sa supériorité en termes de débit par rapport à `kafka-python` lors des pics de charge transactionnelle.
- **Couche de Transport (Streaming) : Redpanda v24.2.26**. Déployé en tant que broker compatible Kafka sans la lourdeur de la JVM, en mode single-node avec Schema Registry et Pandaproxy embarqués. Quatre topics asynchrones sont provisionnés à l'initialisation : `topic_orders`, `topic_order_items`, `topic_order_payments`, `topic_order_reviews`.
- **Couche de Stockage Opérationnel (OLTP) : PostgreSQL 15**. Modélisation en 8 tables normalisées avec contraintes de clés étrangères (FK). Un *healthcheck* SQL strict garantit la disponibilité du moteur avant l'amorçage des consommateurs.
- **Couche de Stockage Décisionnel (OLAP) : ClickHouse**. Utilisation du moteur `MergeTree` optimisé pour les insertions de flux (Append-Only) des tables transactionnelles. Implémentation de dictionnaires en RAM (`COMPLEX_KEY_HASHED`) pour les dimensions, garantissant des jointures logiques en complexité $O(1)$.
- **Couche d'Orchestration (ELT) : Apache Airflow 2.7**. Configuré avec `LocalExecutor` et une base de métadonnées PostgreSQL dédiée. Il orchestre les DAGs de transformation sans jamais charger la donnée en mémoire, déléguant le calcul au moteur ClickHouse via le `ClickHouseOperator`.
- **Couche Machine Learning (MLOps) : scikit-learn** et **Pandas**. Implémentation d'algorithmes de régression (GradientBoosting, RandomForest), de clustering (KMeans) et de détection d'anomalies (IsolationForest).
- **Couche de Visualisation (BI) : Metabase v0.56.1**. Connecté nativement à la couche Gold de ClickHouse pour l'exposition des indicateurs clés de performance (KPIs) en temps réel.

### 4.1.2 Déploiement et Infrastructure as Code

L'ensemble de l'infrastructure distribuée se déploie de manière déterministe en une seule commande grâce à Docker Compose :

```
docker compose up --build -d
```

Le démarrage ordonné et sécurisé des microservices est garanti par l'arborescence des clauses `depends_on` conditionnées par `service_healthy` et `service_completed_successfully`. Le script *Unified Seeder* (initialisation des dimensions) et le *Producer* s'exécutent automatiquement, rendant le pipeline opérationnel *out-of-the-box* sans intervention manuelle.

## 4.2 Implémentation du Producer : Simulation Transactionnelle

Le producer simule un flux d'événements transactionnels réaliste à partir des datasets statiques Olist. L'implémentation garantit des propriétés fondamentales pour les systèmes distribués :

— **Ordre chronologique strict :** Les commandes sont triées par `order_purchase_timestamp` avant publication, assurant la fidélité de la ligne de temps pour les consommateurs en aval.

— **Cohérence transactionnelle (Chunking) :** Pour chaque lot de 100 commandes, le script collecte les entités associées (articles, paiements, avis) via une jointure logique en mémoire, puis les publie dans leurs topics respectifs lors d'un même cycle atomique.

— **Gestion d'état et reprise sur incident :** Un fichier `producer_state.txt` persiste l'offset temporel du dernier événement émis. En cas de crash du conteneur, le producer reprend exactement là où il s'est arrêté (concept de *Checkpointing*).

— **Fiabilité de livraison maximale :** La configuration `acks="all"` exige que tous les nœuds du cluster Redpanda accusent réception avant de poursuivre.

## 4.3 Implémentation des Consumers : Routage et Persistance

### 4.3.1 Consumer ClickHouse (Voie OLAP)

Le consumer analytique implémente un pattern de *Micro-Batching* pour contourner le problème des insertions unitaires, particulièrement anti-performantes dans les bases orientées colonnes :

— **Bufferisation en mémoire :** Les événements sont accumulés dans des dictionnaires Python distincts par topic.

— **Flush conditionnel bidimensionnel :** L'insertion vectorisée (`client.insert_df()`) est déclenchée soit par l'atteinte d'un seuil volumétrique (500 lignes), soit par un timeout temporel (10 secondes), garantissant une latence maîtrisée.

— **Sémantique *At-Least-Once* :** Les offsets Kafka ne sont validés (*commit*) qu'après la confirmation d'écriture réussie par ClickHouse.

— **Garde-fous de typage :** Un dictionnaire `SCHEMA` impose un casting rigoureux avant l'insertion pour respecter les contrats de données stricts de l'engine `MergeTree`.

### 4.3.2 Consumer PostgreSQL (Voie OLTP)

Le consumer relationnel fait face à la complexité des contraintes d'intégrité référentielle (ACID) inhérentes à PostgreSQL :

— **Ordonnancement *FK-Safe* :** La lecture des topics est séquentielle et hiérarchisée : `orders` → `items` → `payments` → `reviews`, empêchant les erreurs de clés orphelines.

— **Idempotence par conception :** Toute insertion utilise la clause `ON CONFLICT (id) DO NOTHING`, immunisant la base contre la duplication de données en cas de rejeu du flux Kafka.

— **Pattern *Dead-Letter Queue* (DLQ) :** Les messages subissant un échec d'insertion persistant (après 10 retries exponentiels) sont isolés dans des logs d'erreurs, évitant l'arrêt total du pipeline de consommation.

## 4.4 Implémentation du pipeline ELT (Medallion Architecture)

### 4.4.1 Transformation Bronze vers Silver (Nettoyage)

Orchestré par Airflow, le script `transform_bronze_to_silver.sql` exécute nativement dans ClickHouse la fiabilisation des données brutes :

1. **Dimensions Clients et Vendeurs :** Résolution spatiale par jointure avec `bronze_geolocation`. Utilisation de la fonction d'agrégation `any()` pour unifier les codes postaux rattachés à de multiples coordonnées.
2. **Dimension Produits :** Application d'un `COALESCE` avec la table de traduction pour normaliser les catégories en anglais.
3. **Tables transactionnelles :** Transfert des commandes et paiements, et élagage (*pruning*) des colonnes textuelles lourdes dans la table des avis pour optimiser les performances analytiques.

### 4.4.2 Transformation Silver vers Gold (Modélisation Dimensionnelle)

L'étape finale, `transform_silver_to_gold.sql`, consolide le schéma en étoile. Plutôt que de stocker physiquement des chaînes de caractères répétitives, la table de faits est construite uniquement avec des clés optimisées :

```
INSERT INTO ecommerce_dw.fact_order_items
SELECT
    i.order_id, i.order_item_id,
```

```
    o.customer_id, i.product_id, i.seller_id,
    toYYYYMMDD(o.order_purchase_timestamp) AS purchase_date_key,
    i.price, i.freight_value,
    coalesce(p.payment_value_total, 0),
    coalesce(p.payment_type_main, 'unknown'),
    CAST(round(coalesce(r.review_score, 0)) AS Int32)
FROM ecommerce_dw.silver_order_items i
JOIN ecommerce_dw.silver_orders o ON i.order_id = o.order_id
LEFT JOIN (...) p ON i.order_id = p.order_id
LEFT JOIN (...) r ON i.order_id = r.order_id;
```

L'astuce architecturale réside dans l'utilisation de `toYYYYMMDD()` qui génère une clé entière temporelle. Lors des requêtes BI en aval, Metabase effectuera des appels `dictGet()` sur les dictionnaires chargés en mémoire vive pour récupérer les noms de villes, de catégories ou de jours, assurant des temps de réponse foudroyants.

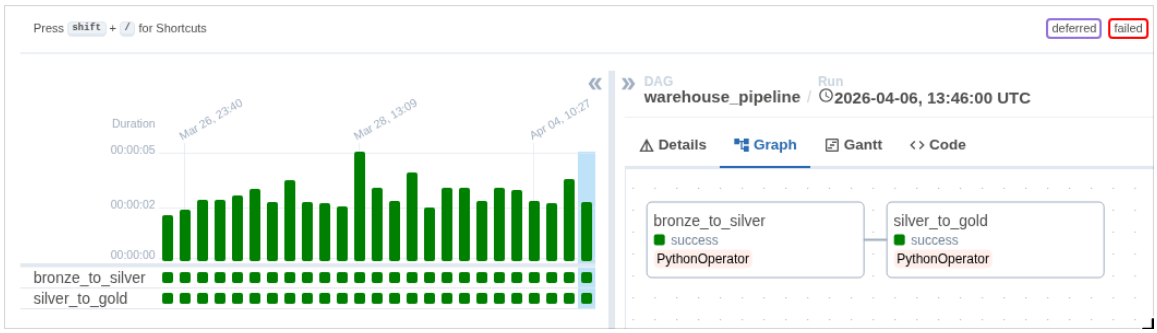


FIGURE 4.1 – Vue du DAG Airflow — pipeline d'ingestion des ventes

## 4.5 Réalisation de la partie Machine Learning

La réalisation de la partie *Machine Learning* s'est appuyée sur une démarche progressive, allant du chargement des données brutes jusqu'à la génération de résultats exploitables dans l'environnement décisionnel. Cette réalisation a nécessité la mise en place d'un pipeline structuré comprenant le chargement des données, la création de variables, l'entraînement des modèles, l'évaluation des résultats et l'intégration finale dans le data warehouse.

### 4.5.1 Chargement et préparation des données

La première étape de réalisation a consisté à charger les différentes tables du dataset Olist utilisées dans le projet. Les fichiers relatifs aux commandes, paiements, clients, produits et avis ont été importés dans l'environnement de travail. Cette phase est essentielle, car elle fournit la matière première nécessaire à tous les traitements analytiques.

Une fois les fichiers chargés, un travail de préparation a été effectué afin d'uniformiser les formats et de rendre les données exploitables. Les colonnes temporelles ont notamment été converties en format date, ce qui est indispensable pour les traitements chronologiques. De plus, les différentes tables ont été reliées entre elles à l'aide de jointures adaptées, afin de constituer des jeux de données cohérents selon les besoins de chaque tâche.

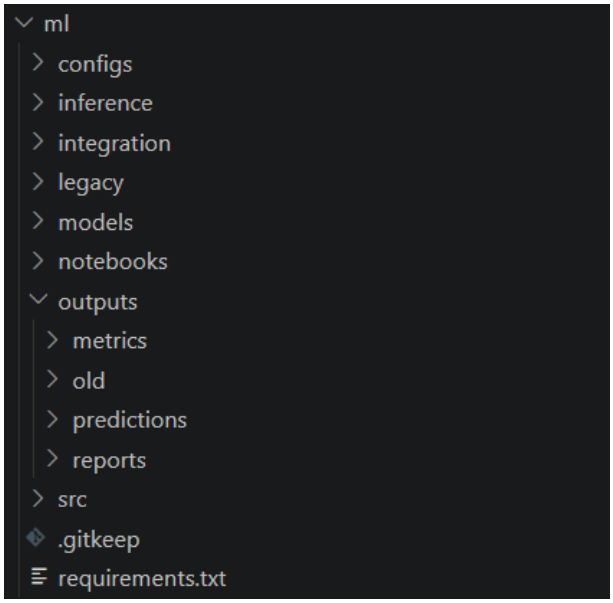


FIGURE 4.2 – l'arborescence du dossier

### 4.5.2 Réalisation du *feature engineering*

Après la préparation initiale, une étape de *feature engineering* a été mise en place afin de produire les variables nécessaires aux modèles. Cette étape représente un élément central de la réalisation, car elle permet de transformer les données brutes en représentations pertinentes pour l'apprentissage automatique.

Dans le cas de la prévision des ventes, seules les commandes livrées ont été retenues, puis fusionnées avec les informations de paiement afin d'obtenir le revenu effectivement payé. Les données ont ensuite été agrégées par mois pour produire une série de chiffre d'affaires mensuel. À partir de cette série, plusieurs variables de retard ont été générées, ainsi que des indicateurs de temps permettant au modèle de mieux capturer la structure temporelle.

Pour la segmentation, les ventes ont été regroupées par client afin de calculer les indicateurs de récence, de fréquence, de montant total dépensé et de panier moyen. Ces variables ont ensuite été standardisées afin de garantir que chaque dimension contribue de manière équilibrée à l'algorithme de clustering.

Pour la détection des anomalies, les données ont été agrégées par jour afin de produire une série journalière du revenu. Ce jeu de données constitue la base des méthodes de détection utilisées ensuite.

| month | month_date | revenue | time_idx | month_num | model_name | predicted_revenue |
|---|---|---|---|---|---|---|
| 2018-05 | 2018-05-01 | 1128836.69 | 18 | 5 | linear_regression | 1281996.8329639516 |
| 2018-06 | 2018-06-01 | 1012090.68 | 19 | 6 | linear_regression | 1360980.9910822574 |
| 2018-07 | 2018-07-01 | 1027903.86 | 20 | 7 | linear_regression | 1507672.7361408735 |
| 2018-08 | 2018-08-01 | 985414.28 | 21 | 8 | linear_regression | 1585902.1281189267 |
| 2018-05 | 2018-05-01 | 1128836.69 | 18 | 5 | random_forest | 1039102.2420640737 |
| 2018-06 | 2018-06-01 | 1012090.68 | 19 | 6 | random_forest | 1038232.5872474071 |
| 2018-07 | 2018-07-01 | 1027903.86 | 20 | 7 | random_forest | 1059601.7467101847 |
| 2018-08 | 2018-08-01 | 985414.28 | 21 | 8 | random_forest | 1038724.2517693118 |
| 2018-05 | 2018-05-01 | 1128836.69 | 18 | 5 | gradient_boosting | 1132237.463171311 |
| 2018-06 | 2018-06-01 | 1012090.68 | 19 | 6 | gradient_boosting | 1081760.6005029327 |
| 2018-07 | 2018-07-01 | 1027903.86 | 20 | 7 | gradient_boosting | 1076385.1519920912 |
| 2018-08 | 2018-08-01 | 985414.28 | 21 | 8 | gradient_boosting | 1082095.3006521466 |

FIGURE 4.3 – les variables ajouter dans la partie Prévision des ventes

### 4.5.3 Réalisation du module de prévision des ventes

La mise en œuvre du module de prévision des ventes a consisté à tester plusieurs modèles de régression sur le jeu de données mensuel préparé précédemment. Parmi les modèles expérimentés figurent la régression linéaire, *Random Forest* et *Gradient Boosting*. Ce choix permet de comparer un modèle simple et interprétable à des modèles plus puissants capables de capturer des relations non linéaires.

Afin de respecter la nature temporelle des données, la séparation entre données d'entraînement et données de test a été réalisée de manière chronologique. Les modèles ont ensuite été évalués à l'aide de plusieurs métriques, notamment le *MAE*, le *RMSE* et le coefficient $R^2$. Le meilleur modèle a été sélectionné sur la base de ses performances globales, puis utilisé pour produire des prévisions sur les périodes futures.

Les résultats obtenus ont été sauvegardés sous forme de fichiers réutilisables, comprenant le modèle entraîné, les métadonnées et les prédictions générées.

| model_name | mae | rmse | r2 |
|---|---|---|---|
| random_forest | 50221.05341570746 | 56085.603261518954 | -0.06734115973704635 |
| gradient_boosting | 54558.25157962035 | 64348.889867290745 | -0.4050202519729731 |
| linear_regression | 395576.7945765023 | 428936.5654766997 | -61.42903277590817 |

FIGURE 4.4 – Comparaison des modeles pour prévision des ventes

### 4.5.4 Réalisation du module de segmentation des clients

Pour la segmentation des clients, plusieurs configurations de l'algorithme *KMeans* ont été testées avec différents nombres de clusters. Cette étape permet de rechercher la structure de regroupement la plus adaptée aux données. Chaque configuration a été évaluée à l'aide du *silhouette score*, qui mesure la qualité de séparation entre les groupes obtenus.

Une fois la meilleure configuration sélectionnée, chaque client a été affecté à un cluster correspondant à son profil comportemental. Les résultats ont ensuite été synthétisés sous forme de tables récapitulatives indiquant les caractéristiques moyennes de chaque segment. Cette étape permet de faciliter l'interprétation métier des groupes identifiés.

Comme pour le module précédent, les objets nécessaires à la réutilisation du processus, notamment le modèle et le *scaler*, ont été sauvegardés.

| model_name | n_clusters | silhouette_score | inertia |
|---|---|---|---|
| kmeans_3 | 3 | 0.7180140559042736 | 196808.26282563253 |
| kmeans_5 | 5 | 0.4678112734064992 | 104964.29819271372 |
| kmeans_4 | 4 | 0.44758301203781325 | 135050.69331916666 |
| kmeans_6 | 6 | 0.3914610344275958 | 90880.40878864093 |

FIGURE 4.5 – Comparaison des modeles pour segmentation des clients

### 4.5.5 Réalisation du module de détection des anomalies

La réalisation du module de détection des anomalies a reposé sur la comparaison de deux approches. La première, basée sur le *Z-score*, mesure l'écart entre la valeur du revenu journalier et la moyenne globale, exprimé en nombre d'écarts-types. Cette méthode permet d'identifier rapidement les observations très éloignées du comportement moyen.

La seconde approche repose sur *Isolation Forest*, un algorithme non supervisé conçu pour isoler les observations rares. Dans le cadre du projet, ce modèle a été utilisé pour repérer les journées de ventes présentant un comportement inhabituel. Après comparaison des résultats, *Isolation Forest* s'est révélé plus pertinent pour la version finale du projet et a été retenu comme modèle principal pour cette tâche.

Les journées détectées comme anormales ont été exportées dans des fichiers dédiés, ce qui permet ensuite de les analyser dans le cadre des tableaux de bord ou d'investigations complémentaires.

| model_name | total_points | num_anomalies | anomaly_ratio |
|---|---|---|---|
| z_score | 611 | 0 | 0.0 |
| isolation_forest | 611 | 13 | 0.02127659574468085 |

FIGURE 4.6 – Comparaison des modeles pour détection des anomalies

### 4.5.6 Sauvegarde des modèles et intégration dans le data warehouse

La dernière étape de réalisation a consisté à organiser les sorties de la partie *Machine Learning* de manière à les rendre exploitables dans l'écosystème global du projet. Les modèles entraînés, les métadonnées, les prédictions, les segments clients et les anomalies détectées ont été sauvegardés dans des fichiers structurés.

En parallèle, une logique d'intégration dans *ClickHouse* a été prévue afin d'insérer les résultats de la prévision, de la segmentation et de la détection des anomalies dans des tables dédiées. Cette intégration permet de relier directement la couche *Machine Learning* au data warehouse décisionnel. Ainsi, les résultats ne restent pas isolés dans un environnement expérimental, mais deviennent des éléments exploitables par les outils de visualisation et d'aide à la décision.

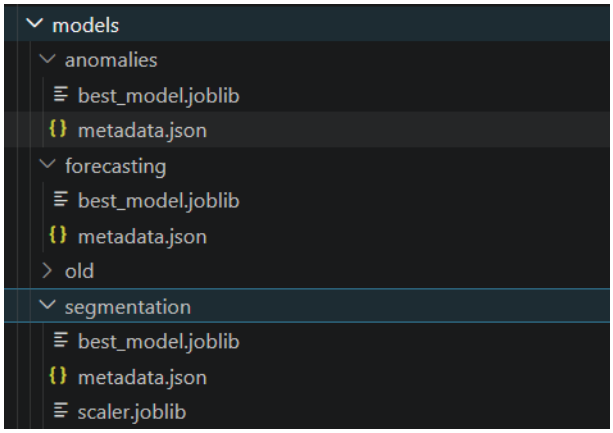


FIGURE 4.7 – Sauvegarde des modeles

### 4.5.7 Bilan de la réalisation

Au final, la partie *Machine Learning* a été réalisée comme une extension cohérente de l'architecture data du projet. Elle permet non seulement d'exploiter les données historiques pour produire des analyses avancées, mais aussi d'enrichir la plateforme avec des

capacités de prévision, de segmentation et de détection d'événements atypiques. Cette réalisation montre que le projet ne se limite pas à une chaîne technique de traitement de données, mais qu'il intègre également une dimension d'intelligence analytique au service de la décision.

## 4.6 Visualisation et Restitution (Metabase)

Metabase exploite la vélocité de ClickHouse via des questions SQL natives. Les dashboards sont structurés en trois niveaux de granularité.

### 4.6.1 Vue Globale (Executive)

Synthèse destinée à la direction, affichant les indicateurs clés (Revenu total, panier moyen, marge estimée) et les dynamiques de croissance (MoM).

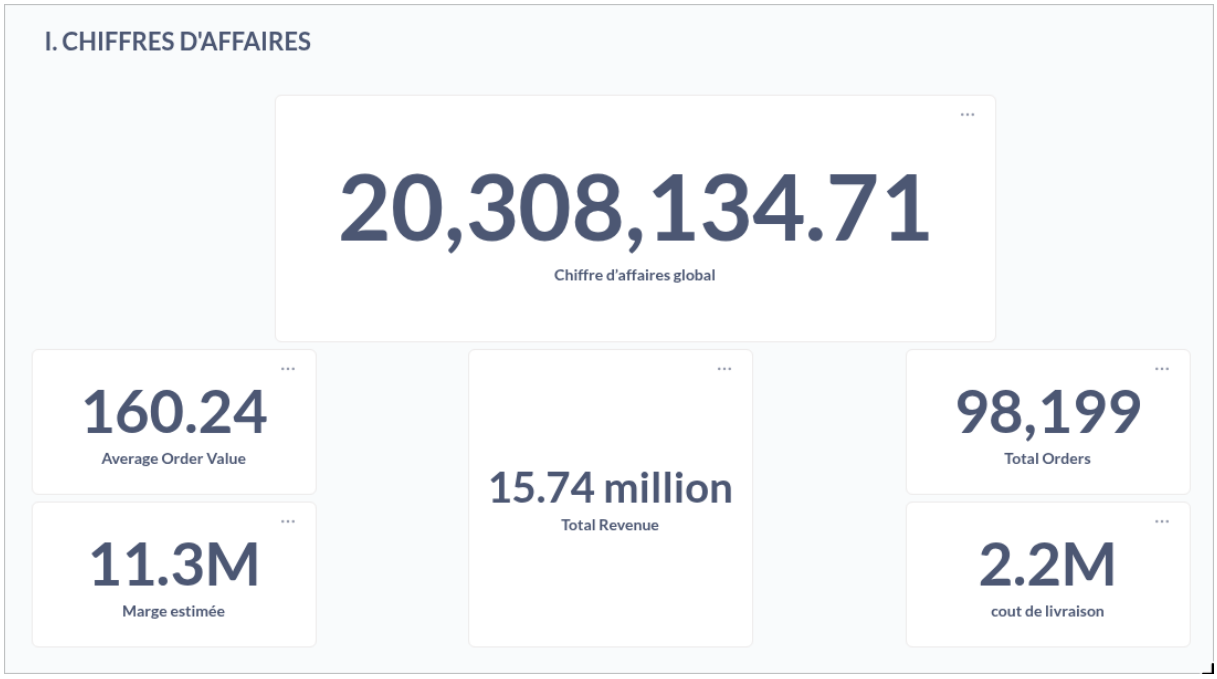


FIGURE 4.8 – Chiffres d'affaires : indicateurs clés (CA global, panier moyen, marge estimée, coût de livraison)

FIGURE 4.9 – Croissance mois par mois : évolution du revenu et du taux de croissance (MoM)

### 4.6.2 Performance des Ventes

Analyse granulaire des revenus par catégories de produits traduites, cartes thermiques de la distribution par État brésilien, et ratios de conversion par types de paiements.

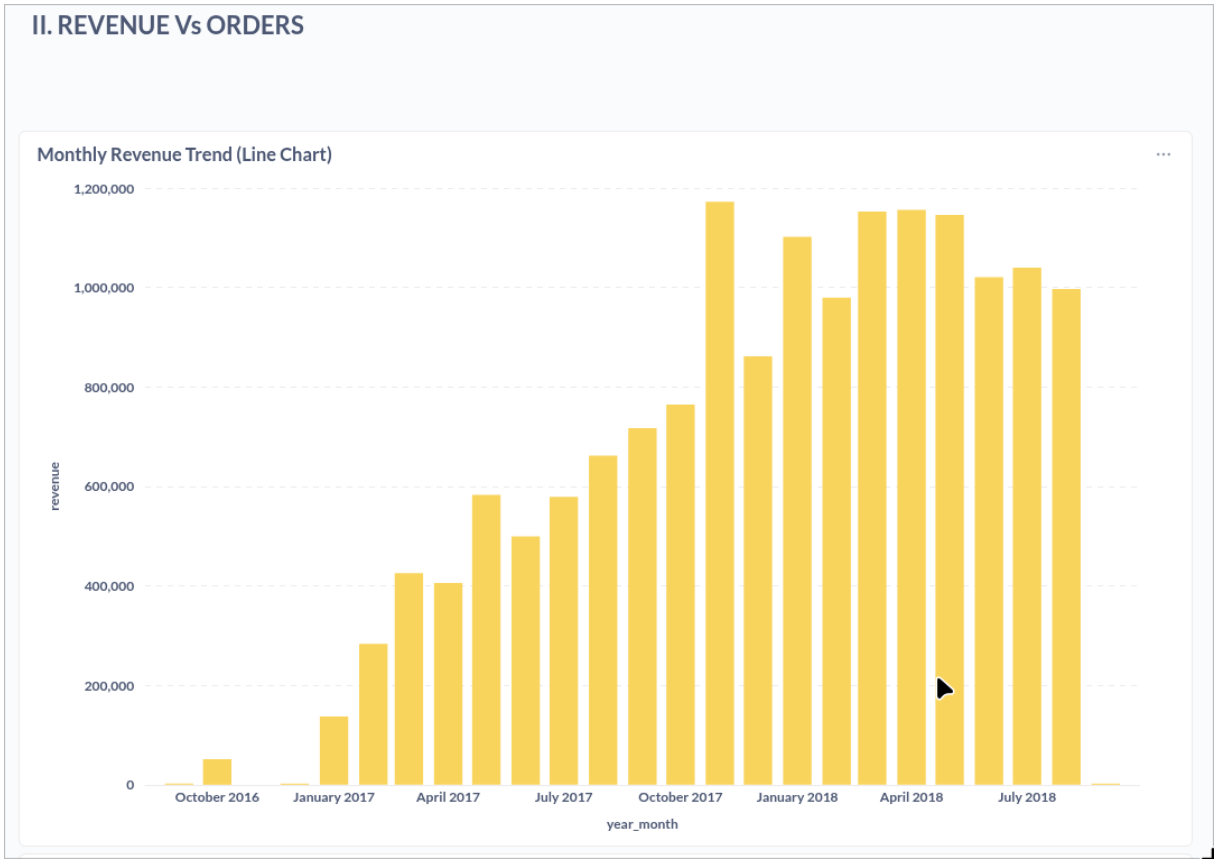


FIGURE 4.10 – Tendance mensuelle du revenu (Monthly Revenue Trend)

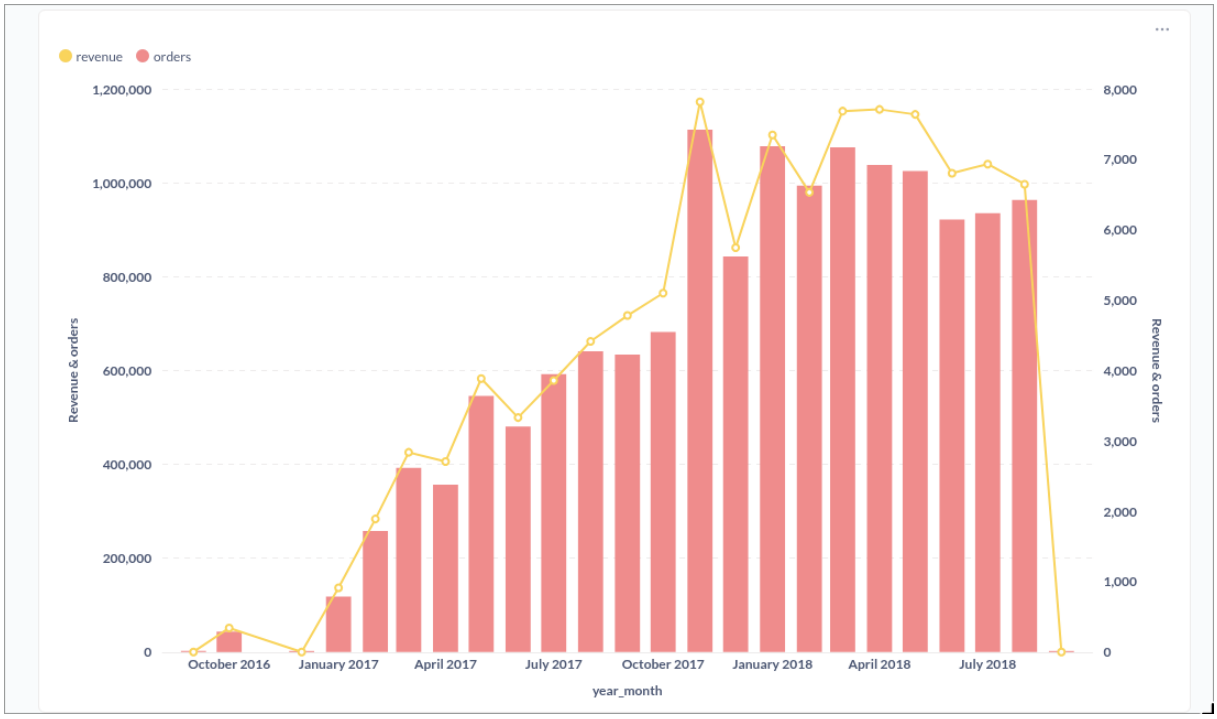


FIGURE 4.11 – Revenu vs nombre de commandes : évolution mensuelle comparative

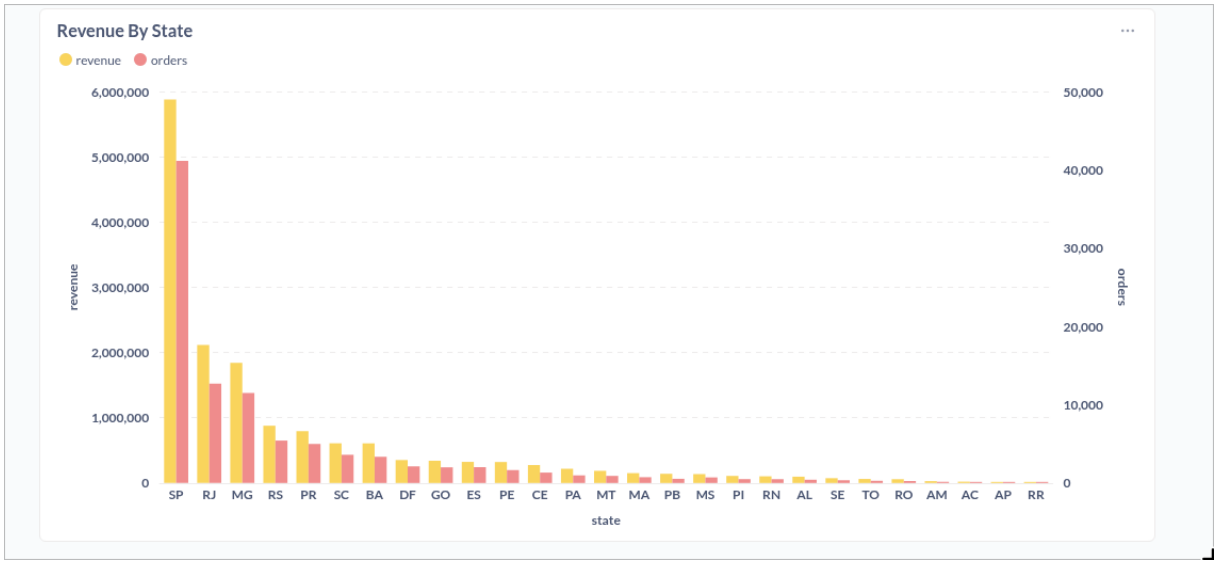


FIGURE 4.12 – Distribution du revenu et des commandes par État brésilien

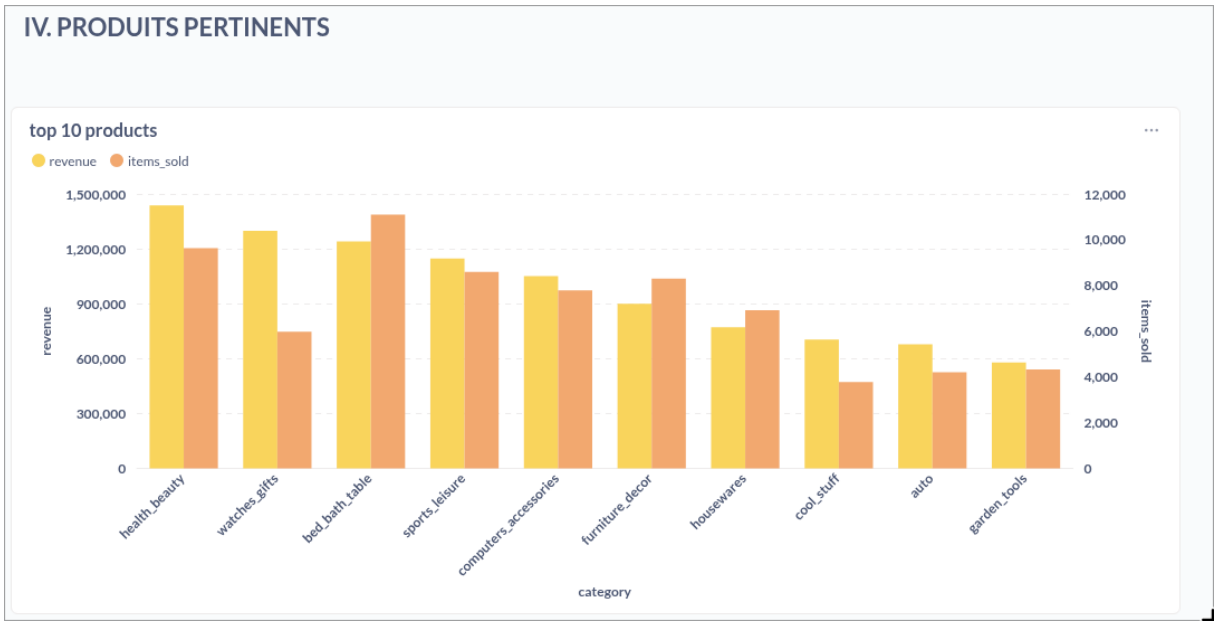


FIGURE 4.13 – Top 10 des catégories de produits par revenu et volume de ventes

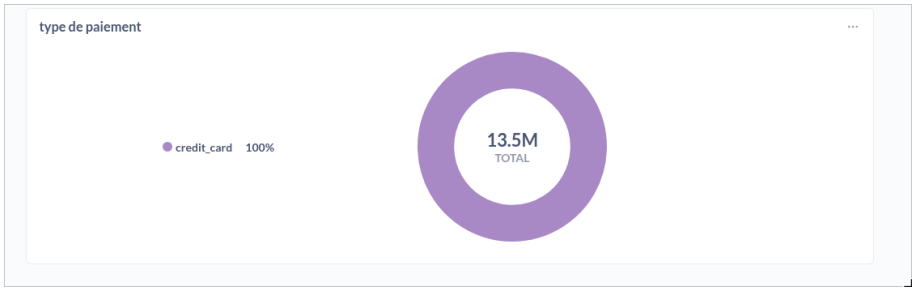


FIGURE 4.14 – Répartition du revenu par type de paiement

### 4.6.3 Performance Opérationnelle

Suivi strict de la chaîne logistique : taux de livraison dans les SLA (*Service Level Agreements*), coûts de fret régionaux, et analyse comportementale de la demande (week-end vs semaine).

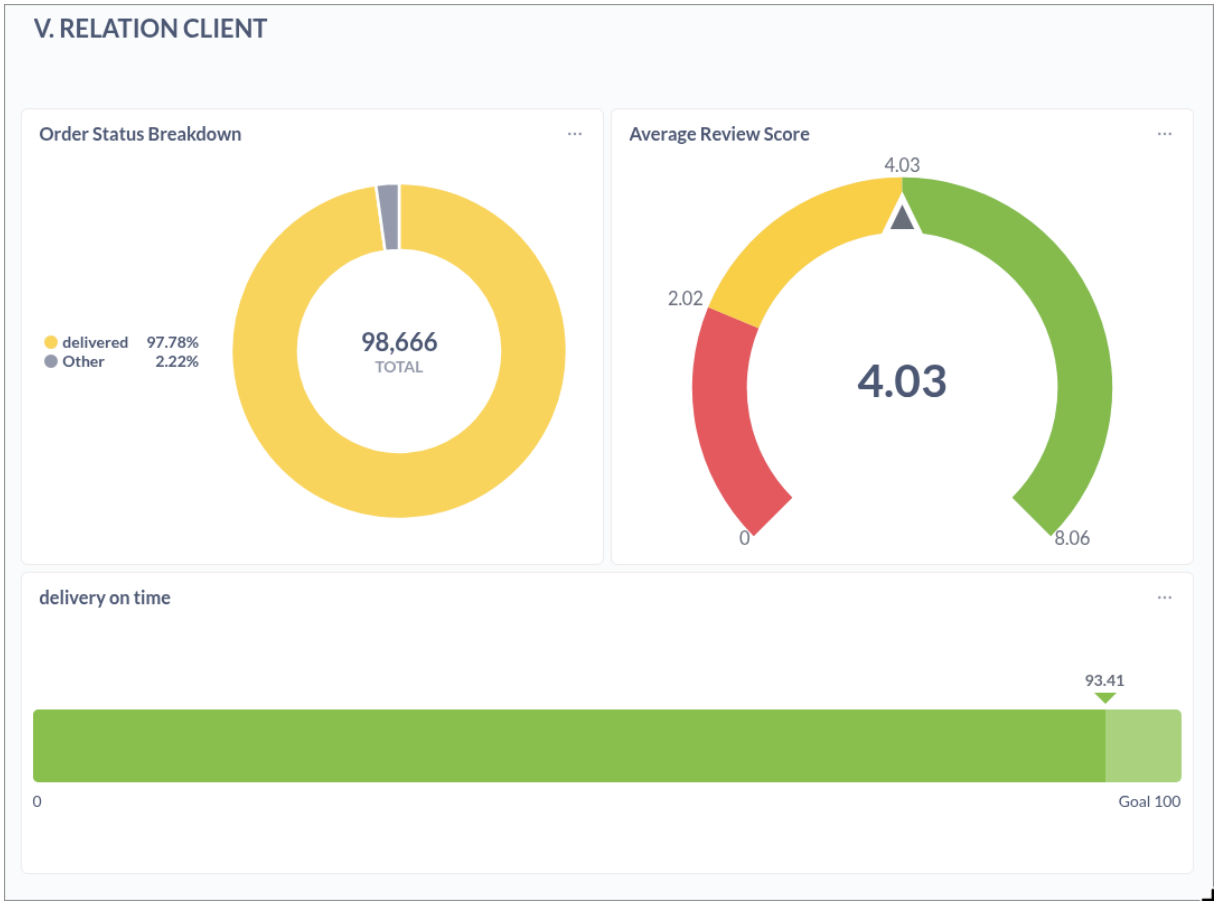


FIGURE 4.15 – Relation client : statut des commandes, score moyen des avis et taux de livraison dans les délais

FIGURE 4.16 – Comportement client : répartition des achats semaine vs week-end et distribution par jour

### 4.6.4 Limitations identifiées et plan de remédiation

L'évaluation critique du système met en exergue des axes d'optimisation pour une mise en production à grande échelle :

| **Limitation architecturale** | **Amélioration recommandée** |
|---|---|
| Transformations SQL "Full-Replace" (TRUNCATE) | Transition vers des modèles incrémentaux (Materialized Views ClickHouse) ou gestion par watermark dans Airflow. |
| Absence de contraintes entre Bronze et Silver | Intégration d'un outil de Data Quality (ex : Great Expectations) dans le DAG Airflow. |
| Entraînement ML manuel | Automatisation du réentraînement (CI/CD pour le ML) via des tâches Airflow planifiées. |
| Gestion des secrets | Externalisation des variables sensibles (DB_PASS) vers un *Secret Manager* ou un fichier `.env` ignoré par Git. |
| Observabilité aveugle du broker | Déploiement d'une stack Prometheus/Grafana pour alerter sur le *Consumer Lag* de Redpanda. |

TABLE 4.1 – Bilan critique et feuille de route technique

# Bibliographie

[1] Bensmina-Anass. streaming-sales-analytics. `https://github.com/Bensmina-Anass/streaming-sales-analytics`, 2025. Consulté le 6 avril 2026.

# Conclusion Générale

Ce projet a permis de concevoir et de réaliser une plateforme de Data Warehouse streaming complète, allant de la simulation d'événements transactionnels jusqu'à la visualisation interactive et l'analyse prédictive par Machine Learning.

Sur le plan technique, le projet valide plusieurs choix architecturaux structurants. L'utilisation de Redpanda comme broker Kafka-compatible offre une alternative légère et performante pour le développement local tout en restant compatible avec les déploiements Kafka en production. L'architecture en médaillon implémentée dans ClickHouse garantit une séparation claire des responsabilités entre ingestion brute, nettoyage et modélisation analytique. La couche Gold en schéma en étoile, avec ses dictionnaires haute performance, démontre une maîtrise des patterns d'optimisation propres aux moteurs OLAP.

Le module Machine Learning constitue la valeur ajoutée différenciatrice du projet : la prévision des ventes par Gradient Boosting, la segmentation client RFM par K-Means et la détection d'anomalies par Isolation Forest forment un écosystème analytique cohérent dont les résultats sont réintégrés dans ClickHouse pour être exploités directement dans les dashboards Metabase.

Plusieurs axes d'amélioration ont été identifiés : l'intégration de dbt pour la gestion versionnée des transformations SQL, l'ajout d'une couche de qualité des données entre Bronze et Silver, le câblage du module ML dans les DAGs Airflow pour un réentraînement automatique, et le déploiement d'une solution d'observabilité Prometheus/Grafana pour le monitoring en production.

Ce projet constitue une démonstration concrète d'une architecture data moderne, résiliente et évolutive, posant des bases solides pour un déploiement en environnement de production.

**L'intégralité du code source est accessible sur le dépôt GitHub du projet** [1].